

Program Generation via Reinforcement Learning with Enumeration Warm-Start

Yiding Song

March 24, 2026

Problem Statement

We are interested in the following problem. Suppose we have access to:

- A set F of functions $f_1, f_2, \dots, f_n$ with types $\tau_1 \rightarrow v_1, \tau_2 \rightarrow v_2, \dots, \tau_n \rightarrow v_n$, where we can query the function implementation (i.e. $f_i(x)$ can be computed for arbitrary well-typed inputs x) but not necessarily the function description (e.g. we do not know the mathematical or programmatic form of the functions).
- A reward function $R : \text{program} \rightarrow \mathbb{R}$ that maps a program to a real-valued reward, where a program is a well-typed composition of functions from F (e.g. this can reward a function for desirable properties such as being non-constant).

How do we synthesise a large number of programs that are diverse and achieve high rewards?

This document describes a two-phase program synthesis solution to the problem: bottom-up enumeration followed by reinforcement learning.

Overview

The pipeline has two phases:

1. **Bottom-up enumeration** — exhaust all programs up to a certain size, quotienting out semantic duplicates (as measured by function behaviour on a test suite), creating a corpus of behaviourally diverse programs.
2. **RL exploration** — a neural policy learns from the corpus (behavioural cloning warm-start), then generates new programs via policy gradient, using a priority queue buffer of the best programs found so far (Abolafia et al., 2018).

1 The Program Grammar

Programs are expressions in a simply-typed lambda calculus extended with conditionals and a fixed set of primitive functions. The expression language is:

$$e ::= c \mid x \mid f(e_1, \dots, e_k) \mid \lambda(x_1, \dots, x_k).e \mid \text{if } e_1 \text{ then } e_2 \text{ else } e_3$$

where c ranges over literals (integers, booleans, empty lists), x over variables, and f over the primitive functions in F .

1.1 Types

The base types are `int` and `bool`. From these, list types `list[ $\sigma$ ]` and function types $\sigma \rightarrow \tau$ are formed. In the current instantiation the concrete types in use are:

`int, bool, list[int], list[bool], list[list[int]]`

All programs synthesised by the pipeline have type `list[int]  $\rightarrow$  list[int]`.

1.2 Polymorphic Functions

Some primitives in F are **polymorphic**: their types contain type variables that can be instantiated to any concrete type. For example:

$\text{map} : (T_1 \rightarrow T_2) \rightarrow \text{list}[T_1] \rightarrow \text{list}[T_2]$

Here T_1 and T_2 are type variables. A **type substitution** θ maps each type variable to a concrete type; applying θ to the signature of `map` yields a monomorphic instantiation. For instance, $\theta = \{T_1 \mapsto \text{int}, T_2 \mapsto \text{bool}\}$ gives:

$\text{map}_\theta : (\text{int} \rightarrow \text{bool}) \rightarrow \text{list}[\text{int}] \rightarrow \text{list}[\text{bool}]$

The set of valid substitutions for a function is determined by the concrete types in use. Each instantiation is treated as a separate monomorphic function during enumeration.

1.3 Higher-Order Functions

A function is **higher-order** if one or more of its arguments has a function type. `map` is higher-order: its first argument is a function $T_1 \rightarrow T_2$. Such arguments cannot be looked up directly in the program bank (which stores complete expressions, not open function values); instead they must be synthesised as lambda expressions, as described in the Higher-Order Applications section below.

2 Part 1: Bottom-Up Enumeration

2.1 Size Metric

Every AST node e is assigned an integer size $|e|$ inductively:

Node	Size
Literal, variable, empty list	1
Application ($f\ a_1\ \dots\ a_k$)	$1 + a_1 + \dots + a_k $
Lambda ($\lambda\ (p_1\ \dots\ p_k)\ e$)	$1 + e $
Conditional (if $c\ t\ e$)	$1 + c + t + e $

Enumeration proceeds by increasing size: programs of size s are built exclusively from sub-expressions of size less than s .

2.2 Observational Equivalence and the Program Bank

We remove program duplicates by comparing their behaviour on a test suite. Fix a test suite $T = \{t_1, \dots, t_m\}$ of m concrete inputs. The *fingerprint* of a closed program p of type $\tau \rightarrow v$ is:

$$\Phi(p) = (p(t_1), \dots, p(t_m)) \in (v \cup \{\dagger\})^m$$

where $\dagger$ records a failed evaluation (runtime error or type error). Two programs $p, q : \tau \rightarrow v$ are observationally equivalent if $\Phi(p) = \Phi(q)$.

Open terms and lambda arguments. However, not all programs are closed. To enumerate an expression like $\text{map } (\lambda y. e) x$, the enumerator must synthesise candidate bodies e . These bodies are *open terms*: they may reference y (the lambda parameter) freely, alongside the outer x . Storing them in the bank allows the same body to be reused across different higher-order functions, avoiding redundant work.

In general, for an open term e with free variables $y_1 : \sigma_1, \dots, y_k : \sigma_k$ (beyond the primary input x), the fingerprint is computed over $T \times V_1 \times \dots \times V_k$, where each V_i is a finite set of probe values for type σ_i :

$$\Phi(e) = (e(t, v_1, \dots, v_k))_{(t, v_1, \dots, v_k) \in T \times V_1 \times \dots \times V_k}$$

Two open terms are considered observationally equivalent iff they agree on this full Cartesian product.

The **program bank** $\mathcal{B}$ maps each pair (τ, s) to a set of programs, containing exactly one representative per observational equivalence class:

$$\mathcal{B}[\tau][s] = \{[p]_{\sim} \mid \vdash p : \tau, |p| = s\}$$

Default test suite ($m = 10$):

`[]`, `[0]`, `[3, 1, 2]`, `[1, 1, 1, 1]`, `[5, 4, 3, 2, 1]`,
`[1, 2, 3, 4, 5, 6, 7, 8]`, `[10, -3, 7, 7, 0]`, `[2, 8, 3, 8, 2, 3]`,
`[0, 1, 0, 1, 0]`, `[42]`

2.3 Base Case

The bank is seeded with size-1 expressions:

- **Integer constants** $c \in C_{\text{seed}}$ of type `int`
- **Boolean constants** $\top, \perp$ of type `bool`
- **Empty lists** `[]` of type `list[σ]` for each list type σ
- **Input variable** $x : \text{list}[\text{int}]$ (since all of our programs are of type `list[int] → list[int]`, this is the variable that will be bound to the input list in a program like `length x`)

2.4 First-Order Applications

For a first-order function $f : A_1 \times \dots \times A_k \rightarrow R$ and target size s , the arguments must satisfy $\sum_{i=1}^k s_i = s - 1$ with each $s_i \geq 1$. For every such composition:

$$\mathcal{B}[R][s] \text{ += } \{f(a_1, \dots, a_k) \mid a_i \in \mathcal{B}[A_i][s_i]\} / \sim$$

Polymorphic functions are pre-expanded into all valid monomorphic instantiations (as described in the Grammar section); each instantiation is then enumerated independently using the formula above.

2.5 Higher-Order Applications and Nested Lambdas

When an argument position requires a function type $A_1 \times \dots \times A_k \rightarrow B$, the lambda argument is synthesised inline rather than looked up in the flat bank. Given a size budget b for the lambda, an extended typing context $\Gamma' = \Gamma \cup \{y_i : A_i\}_{i=1}^k$ is formed, and a child bank $\mathcal{B}'$ is enumerated recursively over Γ' up to size $b - 1$. The lambda candidates are then:

$$\{(\lambda (y_1 \dots y_k) e) \mid e \in \mathcal{B}'[B][b - 1]\}$$

Child banks are keyed by (Γ', b) and cached so that the same lambda type appearing as an argument to several functions at the same size is only built once.

Nesting depth. ℓ counts the number of enclosing lambda scopes. Higher-order calls within a child context are only attempted when $\ell \leq \ell_{\max}$ (default $\ell_{\max} = 2$), preventing combinatorial explosion from arbitrarily deep lambda nesting while still enabling patterns with three levels of nesting such as $\text{map } (\lambda y. \text{map } (\lambda z. \text{map } (\lambda w. \dots) z) y) x$.

2.6 Quality Filtering

After enumeration, the corpus is obtained by retaining programs that pass three conditions. Let $\mathcal{F}(p) \subseteq \{1, \dots, m\}$ denote the indices on which p does not crash. A program p is retained iff:

1. $|\mathcal{F}(p)| \geq 3$ (non-crashing on sufficiently many inputs)
2. $|\{p(t_i) \mid i \in \mathcal{F}(p)\}| \geq 2$ (non-constant output)
3. $\frac{|\{p(t_i) \mid i \in \mathcal{F}(p)\}|}{|\mathcal{F}(p)|} \geq \nu_{\min}$ (output variability, default $\nu_{\min} = 0.3$)

2.7 Key Parameters

Parameter	Default	Role
$s_{\max}$	5	Maximum AST size enumerated
$\ell_{\max}$	2	Maximum lambda nesting depth for HOF arguments
C_{seed}	$\{0, 1, \dots, 10\}$	Integer literals available as base-case expressions
$\nu_{\min}$	0.3	Corpus variability threshold
T	10 inputs	Test suite for fingerprinting and deduplication

3 Part 2: Reinforcement Learning

The RL phase trains a neural policy on the enumerated corpus to generate good programs recursively from the root, making one decision per AST node. This is framed as a Markov Decision Process (MDP) over a grammar-constrained action space.

3.1 MDP Formulation

State. A synthesis state s is a tuple $(\tau, \Gamma, f_{\text{par}}, i, \delta, \ell)$:

Component	Description
τ	Required type at the current node
Γ	Typing context: variables in scope with their types
f_{par}, i	Parent function and argument index (contextual features)
δ	Remaining depth budget (decremented at each recursive call)
ℓ	Current lambda nesting depth

Actions. The valid action set $\mathcal{A}(s)$ depends on s :

Action	Condition
Literal integer c	$\tau = \text{int}$
Literal boolean	$\tau = \text{bool}$
Empty list	τ is a list type
Variable y	$y \in \text{dom}(\Gamma)$ with $\Gamma(y) = \tau$
Apply f (instance)	$\delta > 0$; return type of f matches τ ; if f is higher-order, $\ell < \ell_{\text{max}}^{\text{RL}} = 2$
Lambda	τ is a function type
If	$\delta > 0$

Episode. Generation begins from the state $s_0 = (\text{list}[\text{int}] \rightarrow \text{list}[\text{int}], \emptyset, \cdot, \cdot, \delta_{\text{max}}, 0)$ and proceeds by recursively expanding each chosen action into child states:

- Apply $f(A_1, \dots, A_k) \rightarrow R$: recurse on $(\tau \leftarrow A_i, \delta - 1)$ for each i
- Lambda $(\tau_1 \times \dots \times \tau_k \rightarrow \sigma)$: recurse on $(\tau \leftarrow \sigma, \Gamma \cup \{y_i : \tau_i\}, \delta - 1, \ell + 1)$
- If: recurse on $(\text{bool}, \delta - 1)$ then twice on $(\tau, \delta - 1)$

If any recursive call encounters an empty action set, the episode fails and returns no program.

3.2 Reward

Given a completed program p , let $\Phi(p)$ denote its fingerprint on T , and let $\mathcal{K}$ be the set of fingerprints already known (from the enumeration corpus and prior RL discoveries). The reward is:

$$R(p) = \text{var}(\Phi(p)) \times \begin{cases} 1.0 & \text{if } \Phi(p) \notin \mathcal{K} \\ 0.1 & \text{otherwise} \end{cases}$$

where $\text{var}(\Phi(p)) = |\{p(t_i)\}|/m$ is the fraction of distinct outputs. Programs with $|\mathcal{F}(p)| < 3$ or $|\{p(t_i)\}| < 2$ receive $R(p) = 0$ and are discarded.

The novelty multiplier incentivises exploration of behaviours not yet covered, while the 0.1 floor ensures the policy is not penalised for rediscovering known programs.

3.3 Priority Queue Buffer

The buffer follows the priority queue training scheme of Abolafia et al. (2018). $\mathcal{B}_{\text{buf}}$ maintains the top- K programs by reward (default $K = 5000$). It is a bounded min-heap: a new program p is admitted only if $R(p) > R_{\min}(\mathcal{B}_{\text{buf}})$, evicting the current minimum. Duplicate fingerprints are excluded.

Training samples are drawn uniformly from $\mathcal{B}_{\text{buf}}$ with no recency or priority weighting.

3.4 Policy Network

The policy π_θ maps a state s to a distribution over actions via a masked softmax. A network g_θ produces a logit for each action; invalid actions are excluded by setting their logits to $-\infty$ before normalising:

$$\pi_\theta(a \mid s) = \frac{\exp(g_\theta(s)_a)}{\sum_{a' \in \mathcal{A}(s)} \exp(g_\theta(s)_{a'})}, \quad a \in \mathcal{A}(s)$$

where $g_\theta : \mathcal{S} \rightarrow \mathbb{R}^{|\mathcal{A}|}$ is a two-layer feedforward network:

$$g_\theta(s) = W_2 \text{ReLU}(W_1 \text{enc}(s) + b_1) + b_2$$

State encoder. Six features of s are each embedded or projected into $\mathbb{R}^d$, then concatenated and projected back to $\mathbb{R}^d$:

Feature	Encoding
τ	Learned embedding over a type vocabulary
f_{par}	Learned embedding over the function vocabulary
i (arg index)	Learned embedding, clamped to $[0, 7]$
δ	Learned embedding, clamped to $[0, 15]$
ℓ	Learned embedding, clamped to $[0, 3]$
Γ	Linear projection of a per-type variable-count vector

Default $d = 64$, hidden dimension 128. Invalid actions are masked to $-\infty$ before softmax; the valid set $\mathcal{A}(s)$ is cached by $(\tau, \Gamma, \delta > 0, \ell)$.

3.5 Training Phase 1: Behavioural Cloning Warm-Start

Each corpus program p induces a sequence of state-action pairs $\{(s_t, a_t)\}$ by a DFS traversal of its AST, recording at each node the state that a top-down policy would have been in and the action it would have taken. Aggregating over all corpus programs gives a dataset $\mathcal{D}$. The policy is pre-trained by minimising:

$$\mathcal{L}_{\text{BC}}(\theta) = -\mathbb{E}_{(s, a) \sim \mathcal{D}} [\log \pi_\theta(a \mid s)]$$

(50 epochs, batch size 64, Adam at $\eta = 10^{-3}$). Transitions involving actions outside the vocabulary are dropped.

3.6 Training Phase 2: RL Loop

Each iteration alternates between a sampling phase and a gradient phase.

Sampling (32 episodes per iteration): run an episode under π_θ ; compute $\Phi(p)$ and $R(p)$; if $R(p) > 0$, attempt insertion into $\mathcal{B}_{\text{buf}}$; if $\Phi(p)$ is novel, add it to $\mathcal{K}$.

Gradient update (8 steps per iteration): sample a batch from $\mathcal{B}_{\text{buf}}$ and minimise the reward-weighted log-likelihood loss (Abolafia et al., 2018):

$$\mathcal{L}_{\text{RL}}(\theta) = -\mathbb{E}_{p \sim \mathcal{B}_{\text{buf}}} \left[R(p) \sum_t \log \pi_\theta(a_t \mid s_t) \right]$$

No baseline is subtracted. Gradient norms are clipped to 1.0 before the Adam step ($\eta = 10^{-4}$).

3.7 Trajectory Extraction

The dataset $\mathcal{D}$ (used in both BC and RL) is constructed by a DFS that is the inverse of the episode procedure. Given a complete AST, each node yields a state–action pair:

- λ -node $\rightarrow$ action **Lambda**, recurse into body at $\ell + 1$
- Application $f(a_1, \dots, a_k) \rightarrow$ action **Apply** f , recurse into each a_i with $f_{\text{par}} = f$, i set accordingly
- If-node $\rightarrow$ action **If**, recurse into condition then branches
- Leaf $\rightarrow$ the appropriate literal or variable action

For polymorphic functions, the type instantiation is recovered by matching the inferred return type of f against τ in the current state.

References

Daniel A Abolafia, Mohammad Norouzi, Jonathan Shen, Rui Zhao, and Quoc V Le. Neural program synthesis with priority queue training. *arXiv preprint arXiv:1801.03526*, 2018.